Aller plus loin avec la librairie pandas

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. Rappel des opérations de base **3**

A. Rappel des opérations de base 3

B. Exercice : Quiz 6

III. Notions avancées de Pandas **7**

A. Import de fichier JSON 7

B. Fusion de datasets 8

C. Données temporelles ‘timeseries’ 9

D. Exercice : Quiz 10

IV. L’essentiel **11**

V. Auto-évaluation **11**

A. Exercice 11

B. Test 12

Solutions des exercices **12**

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée (en minutes) : 60

Environnement de travail : Python 3.12+, PyCharm Community

Pré requis : bases en Python, bases de la librairie Pandas

Contexte

Dans un cours précédent, nous avons abordé les notions de Pandas. Nous avons vu que les objets fondamentaux de cette librairie sont les Series et les Dataframes. Ce sont ces structures qui contiennent les données.

Nous avons vu comment créer ces objets à partir de données brutes, qu'ils s'agisse de fichiers Excel ou bien de données entrées manuellement, et nous avons vu comment les manipuler.

Dans ce cours, nous commencerons par très brièvement rappeler les notions fondamentales de manipulation des dataframes.

Ensuite, nous verrons comment utiliser d'autres formats d'entrée qu'Excel, notamment le format Json. Nous verrons comment fusionner des datasets ou encore comment utiliser des données temporelles (timeseries).

II. Rappel des opérations de base

A. Rappel des opérations de base

Dans ce cours, nous utiliserons les données reprises dans le dataset ci-dessous :

Kaggle[1]

[cf. archive.zip]

Il s'agit de mesures météorologiques qui s'étendent du début des années 2000 jusqu'en 2024.

Dans l'exemple ci-dessous, nous allons importer ces données, vérifier leur cohérence et tracer quelques graphiques qui nous permettrons de bien comprendre l'objet de ce dataset.

Exemple

Une fois le fichier « *Weather_dataset.csv* » enregistré à la racine de notre projet pyCharm, nous pouvons directement importer celui-ci grâce à la fonction read_csv() de Pandas :

```
1 import pandas as pd
2 import numpy as np
3 import seaborn as sns
4
5 df = pd.read_csv("Weather_dataset.csv")
6 print(df.head())
```

Vous remarquerez que nous avons choisi d'importer directement Numpy et Seaborn. Ce sont des librairies que nous avons déjà abordées, et qui nous seront utiles également dans le cadre de ce cours.

Le code ci-dessus nous confirme que le dataset a bien été importé :

```
                time  temperature  ...  wind_speed_10m (km/h)  wind_direction
0  2000-01-01T00:00         -0.1  ...                     5.9             169
1  2000-01-01T01:00         -0.2  ...                     5.8             173
2  2000-01-01T02:00         -0.1  ...                     6.1             180
```

1 https://www.kaggle.com/datasets/parthdande/timeseries-weather-dataset

```
3  2000-01-01T03:00          -0.2  ...                    6.8              183
4  2000-01-01T04:00          -0.2  ...                    6.8              183
[5 rows x 16 columns]
```

Les colonnes contiennent beaucoup d'informations. Supposons que nous souhaitions tracer l'évolution de la température en fonction du temps. Une fois que nous disposons de notre dataframe, ceci est extrêmement simple.

L'opération ci-dessous est extrêmement longue. En fonction des performances de votre ordinateur, le graphique ne pourrait même pas s'afficher où un message d'erreur vous disant « *mémoire insuffisante* » pourrait même apparaitre.

```python
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("Weather_dataset.csv")
print(df.head())

sns.lineplot(df, x='time', y='temperature')
plt.show()
```

C'est normal, puisque notre dataset contient beaucoup de données. Tracer un graphique est donc une opération qui peut prendre beaucoup de temps. Voyons la taille de notre dataset :

```python
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("Weather_dataset.csv")
print(df.head())
print(df.describe())
```

Voici le résultat de la fonction describe() :

```
         temperature  relative_humidity  ...  wind_speed_10m (km/h)  wind_direction
count  214152.000000      214152.000000  ...          214152.000000   214152.000000
mean       10.247511          75.838531  ...              12.907395      196.418380
std         8.247190          16.608345  ...               6.128445       92.371503
min       -21.600000          14.000000  ...               0.000000        1.000000
25%         4.000000          65.000000  ...               8.300000      117.000000
50%        10.100000          80.000000  ...              12.000000      218.000000
75%        16.400000          90.000000  ...              16.500000      270.000000
max        37.700000         100.000000  ...              60.200000      360.000000
```

Comme on peut le constater, notre dataframe contient 214 152 lignes. On peut donc facilement imaginer pourquoi tracer un tel graphique est consommateur de mémoire système. Pour connaitre le nombre de lignes, nous aurions également pu utiliser la fonction df.count().

Tous droits réservés à STUDI - Reproduction interdite

Cependant, nous avons appris dans l'introduction de Pandas à sélectionner uniquement une partie des données du dataframe. Considérons les 24 premières lignes du dataframe et traçons le graphique de la température :

Exemple

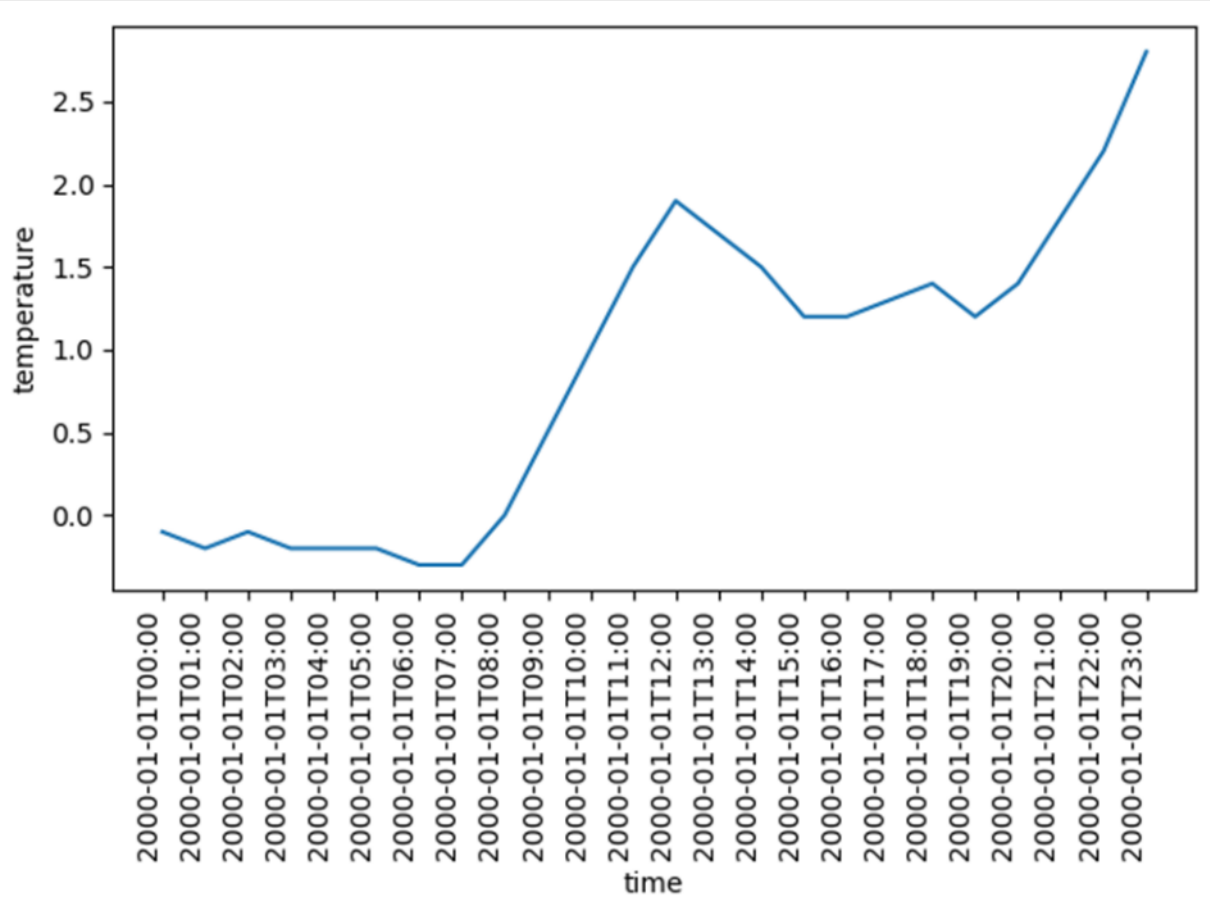
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("Weather_dataset.csv")
print(df.head())
print(df.describe())

dfFiltered = df[0:24]

fig = sns.lineplot(dfFiltered, x='time', y='temperature')
fig.set_xticklabels(fig.get_xticklabels(), rotation=90, ha="right")
plt.tight_layout()
plt.show()
```

On peut remarquer plusieurs choses :

- Lorsqu'on filtre les lignes du dataset (df[0 :24]), il s'agit de prendre l'élément 0 (on commence dès le début du dataset), jusqu'au 23e élément (et non le 24e).
- L'axe des abscisses comporte des étiquettes qui prennent de la place (le temps). Il convient donc ici de faire subir à ces légendes une rotation de 90°, et d'appliquer le tight_layout() afin d'adapter la zone de traçage dans le graphique généré, de manière à pouvoir lire correctement l'ensemble des étiquettes.

Tous droits réservés à STUDI - Reproduction interdite

Le code précédent nous donne donc le graphique suivant :



Qui contient donc 24 points de données (de 00 :00 jusqu'à 23 :00).

B. Exercice : Quiz

[solution n°1 p.13]

Question 1

Pandas ne permet pas de manipuler des grandes structures de données, car tracer des graphiques peut être très lent :

- Vrai
- Faux

Question 2

Si 'df' est un dataframe, combien d'éléments contiendra le dataframe 'df2' lors de l'exécution du code suivant :

```
df2 = df[0:50]
```

- 49
- 50
- 51

Question 3

Tous droits réservés à STUDI - Reproduction interdite

Pour afficher la taille d'un dataframe, je peux utiliser le code suivant :

- ☐ df.describe()
- ☐ df.size
- ☐ df.count()

Question 4

Pour générer un dataframe depuis un fichier CSV, par exemple généré avec Excel, j'utilise la fonction suivante :

- ○ read_csv()
- ○ from_csv()
- ○ import_xls()

Question 5

Pourquoi est-il utile de filtrer les données contenues dans un dataframe ?

- ○ Pour sélectionner uniquement les données qui nous intéressent
- ○ Ce n'est pas utile, Pandas peut très bien travailler sur l'ensemble du dataframe
- ○ Car sinon, Pandas est lent

III. Notions avancées de Pandas

A. Import de fichier JSON

Fichiers JSON

Parfois, les données d'entrée n'existent pas au format .CSV.

Le JSON (Javascript Object Notation) est un format très répandu lorsqu'il s'agit de structurer des données. Ce format de données est basé sur le principe « *clé* » : « *valeur* ».

Un contenu JSON ressemble généralement à ceci :

```
[
{"firstName":"Brad", "lastName":"Pitt"},
{"firstName":"Thomas", "lastName":"Dupont"},
{"firstName":"Tom", "lastName":"Cruise"},
{"firstName":"Bradley", "lastName":"Cooper"}
]
```

Les fichiers JSON ont simplement une extension .json. Voyons comment importer de tels fichiers à l'aide de Pandas :

Exemple

Copiez le contenu de l'extrait JSON ci-dessus dans un fichier texte (par exemple à l'aide du bloc-notes), et enregistrez-le dans le répertoire de travail avec l'extension '.json'. Par exemple, donnez lui le nom « *mon-fichier.json* ».

Tous droits réservés à STUDI - Reproduction interdite

Ensuite, utilisons Pandas pour importer notre fichier, et affichons ce que renvoie la fonction head() :

```
1 import pandas as pd
2
3 import seaborn as sns
4
5 df = pd.read_json("mon-fichier.json")
6 print(df.head())
```

Ce programme vous renvoie ceci :

```
  firstName lastName
0      Brad     Pitt
1    Thomas   Dupont
2       Tom   Cruise
3   Bradley   Cooper
```

On constate que notre fichier JSON a bien été converti en dataframe grâce à Pandas.

B. Fusion de datasets

Fusionner deux datasets permet de compléter les données d'un dataset avec les données de l'autre dataset. Il ne s'agit donc pas simplement de « *concaténer* » les deux datasets, mais bien de définir des colonnes communes entre les deux, de manière à ce qu'il n'y ait pas de colonnes redondantes dans le dataset résultant. Voyons ceci avec un exemple :

Exemple

Reprenons le dataframe résultant de notre exemple précédent, qui était le résultat de l'import d'un contenu au format JSON et complétons le avec l'âge :

```
  firstName lastName  age
0      Brad     Pitt   35
1    Thomas   Dupont   40
2       Tom   Cruise   45
3   Bradley   Cooper   50
```

Supposons par ailleurs que nous disposions d'un dataset similaire, mais qui contient la couleur préférée de chaque personne :

```
  firstName lastName  color
0      Brad     Pitt   blue
1    Thomas   Dupont    red
2       Tom   Cruise  white
3   Bradley   Cooper  green
```

Nous souhaitons fusionner ces deux datasets. Pour ce faire, il existe la fonction pd.merge() :

```
1 import pandas as pd
2
3 df_age = pd.read_json("mon-fichier_age.json")
4 df_color = pd.read_json("mon-fichier_color.json")
5
6 result = pd.merge(df_age, df_color, on=["firstName", "lastName"])
7
```


Tous droits réservés à STUDI - Reproduction interdite

```
print(result.head())
```

Dans cette fonction, on définit les dataframes à fusionner, ainsi que les clés de colonnes à mettre en commun. Le résultat de ce code est le suivant :

```
  firstName lastName  age  color
0      Brad     Pitt   35   blue
1    Thomas   Dupont   40    red
2       Tom   Cruise   45  white
3   Bradley   Cooper   50  green
```

Ce qui est bien le résultat attendu.

C. Données temporelles 'timeseries'

Les structures de données contiennent souvent une variable de temps. Il est possible, avec Pandas, d'interpréter cette donnée non pas comme un texte, mais bien comme une donnée temporelle. C'est ce qu'on appelle une 'timeseries' (littéralement : série temporelle). Voyons un exemple :

Exemple

Le dataset téléchargé au début de ce cours contenait le temps comme première colonne. Lors de l'import du dataset pour en générer un dataframe, le temps est d'abord interprété comme une donnée de texte.

Cependant, il est possible avec Pandas, d'interpréter cette colonne comme une série temporelle :

```
import pandas as pd

df = pd.read_csv("Weather_dataset.csv")
df["time"] = pd.to_datetime(df["time"])

print(df.describe())
```

Nous utilisons donc la fonction 'to_datetime()' de Pandas pour convertir ces données en données temporelles.

Le résultat de l'appel à describe() nous donne ceci :

```
                      time  ...  wind_direction
count                214152  ...   214152.000000
mean    2012-03-19 11:30:00  ...      196.418380
min     2000-01-01 00:00:00  ...        1.000000
25%     2006-02-08 17:45:00  ...      117.000000
50%     2012-03-19 11:30:00  ...      218.000000
75%     2018-04-28 05:15:00  ...      270.000000
max     2024-06-05 23:00:00  ...      360.000000
std                     NaN  ...       92.371503
```

On voit que cette fois, le temps devient bien une donnée interprétable par Pandas. Le minimum correspond à la date la plus ancienne, tandis que le maximum correspond à la date la plus récente.

Tous droits réservés à STUDI - Reproduction interdite

On peut effectuer, par exemple, des opérations de soustraction pour calculer un intervalle de temps :

```
import pandas as pd

df = pd.read_csv("Weather_dataset.csv")
df["time"] = pd.to_datetime(df["time"])

print(df.describe())

print(df["time"].max() - df["time"].min())
```

Ce qui nous donne :

```
8922 days 23:00:00
```

Ce qui représente bien l'intervalle de temps qui caractérise le dataset en question.

Il est également possible de filtrer sur les valeurs de temps. Par exemple, considérons que nous souhaitions uniquement garder les mesures relatives à l'année 2024 :

```
import pandas as pd

df = pd.read_csv("Weather_dataset.csv")
df["time"] = pd.to_datetime(df["time"])

df2024 = df[df["time"] > pd.Timestamp("2024-01-01")]
print(df2024.describe())
```

Ce qui nous donne :

```
                                time  ...  wind_direction
count                           3767  ...     3767.000000
mean   2024-03-19 11:59:59.999999744  ...      184.153969
min              2024-01-01 01:00:00  ...        1.000000
25%              2024-02-09 06:30:00  ...      108.500000
50%              2024-03-19 12:00:00  ...      197.000000
75%              2024-04-27 17:30:00  ...      257.000000
max              2024-06-05 23:00:00  ...      360.000000
std                              NaN  ...       86.374410
```

On constate qu'effectivement, la date minimale est le 01/01/2024. On a donc uniquement gardé l'année 2024.

D. Exercice : Quiz

[solution n°2 p.14]

Question 1

Il est possible d'importer des fichiers JSON directement dans un dataframe :

- ○ Vrai
- ○ Faux

Question 2

Tous droits réservés à STUDI - Reproduction interdite

Que fait la ligne suivante :

```
result = pd.merge(df1, df2, on="ID")
```

- Elle concatène deux dataframes df1 et df2
- Elle fusionne deux dataframes df1 et df2, en considérant la colonne 'ID' comme étant commune
- Ce code génère une erreur, car merge() ne prend pas d'argument 'on'

Question 3

Qu'est ce qu'une 'timeseries' ?

- Une heure au format String
- Un type de graphique
- Une série temporelle (vecteurs de dates)

Question 4

Comment doit-on calculer l'intervalle de temps entre deux variables de type Timestamp ?

- En utilisant simplement l'opérateur de soustraction '-' entre les deux variables
- En utilisant pd.interval_calc()
- Ce n'est pas possible directement

Question 5

Que fait ce code :

```
dfNew = df[df["time"] > pd.Timestamp("2024-01-06")]
```

- Il effectue un filtre de 'df' pour ne conserver que les points dont le contenu de la colonne 'time' est ultérieur au 06 janvier 2024
- Il génère une erreur : le format de temps est incorrect
- Il ne prend que les données qui correspondent au 06 janvier 2024

IV. L'essentiel

La librairie Pandas permet de manipuler efficacement et rapidement de larges structures de données.

Nous avons vu qu'elle est également versatile ; elle peut en effet s'adapter à divers types de formats de données d'entrée, notamment le CSV et le JSON, deux des formats de données les plus répandus.

Nous avons vu comment filtrer des données, comment fusionner des dataframes entre eux, et comment interpréter et exploiter des données temporelles ('timeseries') dans nos dataset.

En ayant complété le cours sur les notions de la librairies Pandas avec celui-ci, vous êtes désormais prêts à utiliser efficacement vos datasets, pour pouvoir réaliser toutes sortes d'analyses de données.

V. Auto-évaluation

A. Exercice

Nous allons analyser l'évolution de la vitesse du vent sur l'année 2022. Pour cela, nous allons à nouveau utiliser le dataset abordé durant ce cours.

Tous droits réservés à STUDI - Reproduction interdite

Question 1

[solution n°3 p.15]

Importez le dataset, et faites en sortes que la colonne 'time' de notre dataframe soit une série temporelle interprétable par Pandas en tant que telle.

Question 2

[solution n°4 p.15]

Tracez l'évolution de la vitesse du vent en fonction du temps pour le mois de janvier de l'année 2022.

B. Test

Exercice 1 : Quiz

[solution n°5 p.16]

Question 1

Pandas traite des larges structures de données efficacement :

- ○ Vrai
- ○ Faux

Question 2

Avec quelles librairies s'emploie généralement Pandas ?

- ☐ Numpy
- ☐ Matplotlib
- ☐ Seaborn

Question 3

Pourquoi l'instruction df.size ne comporte-t-elle pas de parentheses ?

- ○ Elle doit en comporter, c'est une erreur
- ○ Car l'appel de fonctions sans parentheses est toléré en Python
- ○ Car size est un attribut, et non une méthode

Question 4

Si j'ai affaire à un très grand dataset, comment puis-je réduire le temps de traitement lors de sa manipulation ?

- ○ En filtrant les données dès le départ, pour ne sélectionner que les données dont j'ai besoin
- ○ Il faut supprimer les données superflues du dataset de départ (par exemple, dans le fichier .csv)
- ○ On ne pourra pas réduire le temps de traitement

Question 5

Quelle est la différence entre pd.concat() et pd.merge() ?

- ○ Ce sont deux fonctions équivalentes
- ○ concat() permet de concaténer, tandis que merge() permet de fusionner en spécifiant des colonnes comme étant identiques
- ○ pd.concat() agit sur les lignes, tandis que pd.merge() agit sur les colonnes

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 6 Solution n°1

Question 1

Pandas ne permet pas de manipuler des grandes structures de données, car tracer des graphiques peut être très lent :

- ○ Vrai
- ◉ Faux

Lorsqu'on a affaire à de grandes structures de données, on ne génère jamais, ou très rarement, des graphiques sur base des données « *brutes* ».

En effet, un tel graphique, en plus de consommer énormément de ressources, n'apportera pas beaucoup d'informations utiles. Dans ce cas là, les données sont préalablement nettoyées et filtrées.

Cependant, vous avez vu dans les exemples que Pandas est capable, en quelques millisecondes, d'effectuer des opérations sur l'ensemble de notre dataframe. On peut donc dire que Pandas est très bien adaptée pour manipuler de grandes (voir très grandes) structures de données.

Question 2

Si 'df' est un dataframe, combien d'éléments contiendra le dataframe 'df2' lors de l'exécution du code suivant :

```
df2 = df[0:50]
```

- ○ 49
- ◉ 50
- ○ 51

Le dataframe 'df2' contiendra l'élément à l'index 0 du dataframe 'df'. Le dernier élément de 'df2' sera l'élément 49 (et non 50 !) du dataframe 'df'. Donc 'df2' contiendra 49 + 1 = 50 éléments.

Question 3

Pour afficher la taille d'un dataframe, je peux utiliser le code suivant :

- ☑ df.describe()
- ☐ df.size
- ☑ df.count()

df.describe() et df.count() renvoient toutes les deux le nombre d'éléments contenus dans chaque colonne. Attention, df.size (qui est un attribut, pas une fonction), est une variable qui contient le nombre de colonnes multiplié par le nombre de lignes. Il n'est donc pas possible de récupérer le nombre de lignes ou de colonnes du dataframe via cet attribut.

Question 4

Pour générer un dataframe depuis un fichier CSV, par exemple généré avec Excel, j'utilise la fonction suivante :

- ◉ read_csv()
- ○ from_csv()
- ○ import_xls()

Tous droits réservés à STUDI - Reproduction interdite

« *read_csv()* » génère un dataframe directement depuis le contenu du fichier Excel enregistré au format CSV.

Question 5

Pourquoi est-il utile de filtrer les données contenues dans un dataframe ?

- (●) Pour sélectionner uniquement les données qui nous intéressent
- ( ) Ce n'est pas utile, Pandas peut très bien travailler sur l'ensemble du dataframe
- ( ) Car sinon, Pandas est lent

Même si Pandas peut effectivement travailler sur l'ensemble des données, on essaye généralement de se simplifier la vie au maximum. Travailler avec des données préalablement filtrées permet de gagner du temps, d'y voir clair, et de tirer des indicateurs pertinents sur nos données. Par ailleurs, parfois, certaines données sont incohérentes et doivent être « *jetées* ».

Exercice p. 10 **Solution n°2**

Question 1

Il est possible d'importer des fichiers JSON directement dans un dataframe :

- (●) Vrai
- ( ) Faux

Oui, c'est tout à fait possible grâce a la fonction read_json() de la librairie Pandas, à l'instar de ce que fait la fonction read_csv() pour les fichiers .csv.

Question 2

Que fait la ligne suivante :

```
result = pd.merge(df1, df2, on="ID")
```

- ( ) Elle concatène deux dataframes df1 et df2
- (●) Elle fusionne deux dataframes df1 et df2, en considérant la colonne 'ID' comme étant commune
- ( ) Ce code génère une erreur, car merge() ne prend pas d'argument 'on'

Les deux dataframes df1 et df2 sont fusionnés l'un avec l'autre, en considérant que le contenu de la colonne 'ID' est commun aux deux dataframes.

Question 3

Qu'est ce qu'une 'timeseries' ?

- ( ) Une heure au format String
- ( ) Un type de graphique
- (●) Une série temporelle (vecteurs de dates)

Il s'agit d'un vecteur constitué de dates, qui peuvent être interprétées comme telles par Pandas. Il ne s'agit donc pas simplement de texte ou de nombres sans signification.

Question 4

Comment doit-on calculer l'intervalle de temps entre deux variables de type Timestamp ?

Tous droits réservés à STUDI - Reproduction interdite

- ⦿ En utilisant simplement l'opérateur de soustraction '-' entre les deux variables
- ○ En utilisant pd.interval_calc()
- ○ Ce n'est pas possible directement

Il est possible de soustraire simplement deux Timestamps. Dans ce cas, Pandas renverra l'intervalle de temps compris entre les deux timestamps mentionnés dans la soustraction.

Question 5

Que fait ce code :

```
dfNew = df[df["time"] > pd.Timestamp("2024-01-06")]
```

- ⦿ Il effectue un filtre de 'df' pour ne conserver que les points dont le contenu de la colonne 'time' est ultérieur au 06 janvier 2024
- ○ Il génère une erreur : le format de temps est incorrect
- ○ Il ne prend que les données qui correspondent au 06 janvier 2024

Le filtre consiste à ne conserver que les données ultérieures (« *plus grandes que* ») au 06 janvier 2024. 'dfNew' est la version filtrée de 'df'.

p. 12 Solution n°3

Importer le dataset se fait de manière tout à fait classique par l'utilisation de read_csv(), puisque notre dataset nous est proposé au format .csv.

Une fois l'import effectué, il suffit de demander à Pandas de convertir le texte contenu dans la colonne 'time' en série temporelle. Comme nous l'avons vu durant le cours, ceci peut se faire par l'utilisation de la fonction pd.to_datetime() :

```
1 import pandas as pd
2
3 df = pd.read_csv("Weather_dataset.csv")
4 df["time"] = pd.to_datetime(df["time"])
```

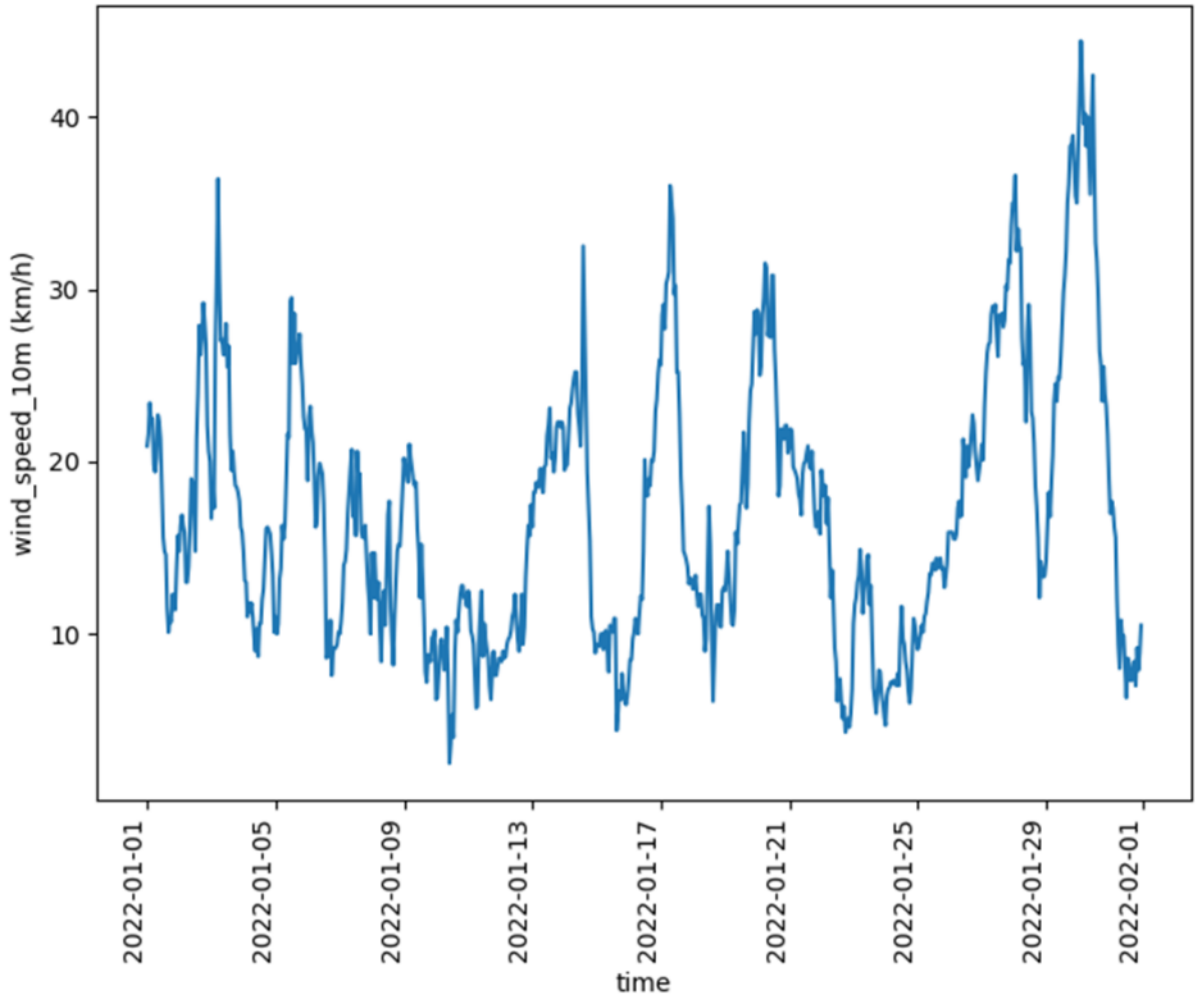
p. 12 Solution n°4

Il est d'abord nécessaire de filtrer nos données, de manière à ne conserver que le mois de janvier 2022. Ensuite, un simple graphique est réalisé. Dans ce cas, nous utiliserons la librairie Seaborn pour tracer ce graphique, même si d'autres librairies, comme Matplotlib ou Plotly peuvent être employées de manière tout à fait équivalente :

```
1 import pandas as pd
2
3 df = pd.read_csv("Weather_dataset.csv")
4 df["time"] = pd.to_datetime(df["time"])
5
6 df2024_01 = df[df["time"] >= pd.Timestamp('2022-01-01')]
7 df2024_01 = df2024_01[df2024_01["time"] < pd.Timestamp('2022-02-01')]
8 print(df2024_01.describe())
```

Tous droits réservés à STUDI - Reproduction interdite

On peut alors tracer le contenu du dataframe à l'aide de Seaborn :



Exercice p. 12 **Solution n°5**

Question 1

Pandas traite des larges structures de données efficacement :

- (●) Vrai
- ( ) Faux

Pandas a été spécialement optimisé pour réduire son temps de traitement. C'est donc une librairie tout à fait capable de manipuler de très grandes structures de données (big data) de manière efficace.

Question 2

Avec quelles librairies s'emploie généralement Pandas ?

- [x] Numpy
- [x] Matplotlib
- [x] Seaborn

Tous droits réservés à STUDI - Reproduction interdite

En réalité, ces 3 librairies sont très souvent employées de manière conjointe. Les 3 propositions sont donc correctes.

Question 3

Pourquoi l'instruction df.size ne comporte-t-elle pas de parentheses ?

- ○ Elle doit en comporter, c'est une erreur
- ○ Car l'appel de fonctions sans parentheses est toléré en Python
- ◉ Car size est un attribut, et non une méthode

Df.size est simplement un attribut de l'objet dataframe 'df', et non une méthode qui lui est liée. Si cela avait été une méthode, l'emploi des parentheses aurait été obligatoire.

Question 4

Si j'ai affaire à un très grand dataset, comment puis-je réduire le temps de traitement lors de sa manipulation ?

- ◉ En filtrant les données dès le départ, pour ne sélectionner que les données dont j'ai besoin
- ○ Il faut supprimer les données superflues du dataset de départ (par exemple, dans le fichier .csv)
- ○ On ne pourra pas réduire le temps de traitement

La manière la plus simple est de filtrer les données le plus tôt possible, pour ne pas alourdir les taches de manipulation du dataframe. Manipuler les fichiers d'entrées (qu'il s'agisse de fichier csv ou json) est généralement assez lourd et source d'erreurs. Il est encore possible de filtrer les données dans l'appel à la fonction pd.read_csv() directement.

Question 5

Quelle est la différence entre pd.concat() et pd.merge() ?

- ○ Ce sont deux fonctions équivalentes
- ◉ concat() permet de concaténer, tandis que merge() permet de fusionner en spécifiant des colonnes comme étant identiques
- ○ pd.concat() agit sur les lignes, tandis que pd.merge() agit sur les colonnes

Merge() permet de fusionner, tandis que concat() permet tout simplement de concaténer, c'est-à-dire « *coller* » les données les unes à la suite des autres.

Tous droits réservés à STUDI - Reproduction interdite